To deal with these sorts of changes, you'll need to refactor code. Without a good set of specs, refactoring is a daunting task. It's impossible to predict how much code you'll need to rework for any given change.

Our challenge as developers is to structure our projects so that big changes are easy and predictable. As Kent Beck says, "for each desired change, make the change easy (warning: this may be hard), then make the easy change."[1]

Your specs will help you toward this goal. They provide a safety net and guard against regressions. They also point out places where the code is too tightly coupled—where you'll need to work harder to "make the change easy."

Guiding Design

If you write your specs before your implementation, you'll be your own first client. You'll get a feel for what it's like to use the interfaces you're building. A good suite of examples will guide the initial design and will support the refactoring as your design evolves.

As counterintuitive as it may sound, one of the purposes of writing specs is to cause pain—or rather, to make poorly designed code painful. By surfacing the pain of a design problem early, specs allow you to fix it while it's cheap and easy to do so.

If you find yourself wrestling a bunch of ungainly objects into place just to test a single method, what are the odds your team is going to be able to use that method correctly every time?

Sustainability

When you drive your code with RSpec, it may take a little extra time to build your first feature. With each new feature you add, however, you'll gain consistent productivity.

Without RSpec or a similar tool in your arsenal, later features tend to take much longer than the first. You're constantly fighting with existing code to make sure it continues working.

This benefit doesn't apply to all projects. If you're writing a throwaway project, or an app with a tiny, frozen feature set, testing extensively at every layer might be overkill.

1. https://twitter.com/kentbeck/status/250733358307500032